Icono *Training Consulting*

Formación

Curso Introducción a Git

2026

Formador

Ana Isabel Vegas

INGENIERA INFORMÁTICA con Master Máster Universitario en Gestión y Análisis de Grandes Volúmenes de Datos: Big Data, tiene la certificación PCEP en Lenguaje de Programación Python y la certificación JSE en Javascript. Además de las certificaciones SCJP Sun Certified Programmer for the Java 2 Platform Standard Edition, SCWD Sun Certified Web Component Developer for J2EE 5, SCBCD Sun Certified Business Component Developer for J2EE 5, SCEA Sun Certified Enterprise Architect for J2EE 5.

Desarrolladora de Aplicaciones FULLSTACK, se dedica desde hace + de 20 años a la CONSULTORÍA y FORMACIÓN en tecnologías del área de DESARROLLO y PROGRAMACIÓN.

training@iconotc.com

Introducción a GIT

❑ **Duración:** 16 horas

❑ **Modalidad:** On-line

❑ **Fechas/Horario:**

- Días 7, 8, 9 y 10 de Abril de 2026
- Horario 9:30 – 13:30 hs.

❑ **Contenidos:**

- Conceptos básicos
  - ¿Qué es y que no es GIT?
  - Cómo usar correctamente GIT
  - Configuración imprescindible
- Fundamentos y arquitectura interna de GIT
  - Diferencias entre el working directory, el stage y el repositorio
  - Qué es exactamente un commit
  - Tipos de referencias (HEAD, ramas y tags)
- Commits
  - Cómo preparar selectivamente un commit
  - Cuando hacer un commit
  - Cómo deshacer commits
  - Buenas prácticas
- Como resolver los conflictos
  - Qué es un conflicto, cómo reducir su frecuencia y cómo resolverlos
- Branching
  - Qué es exactamente una rama en GIT
  - Cómo trabajar con ramas y diferentes maneras de fusionarlas
  - Modelos de branching
  - Git Flow
- Trabajando con repositorios remotos
  - Cómo comunicar los repositorios locales con los remotos
  - Diferentes escenarios cuando trabajamos en equipo contra un repositorio remoto
  - Configuración básica
- Pull Requests
  - Por qué usar PR
  - Cómo hacer correctamente una PR
  - Cómo configurar el repositorio remoto si usamos “PR”

Conceptos básicos

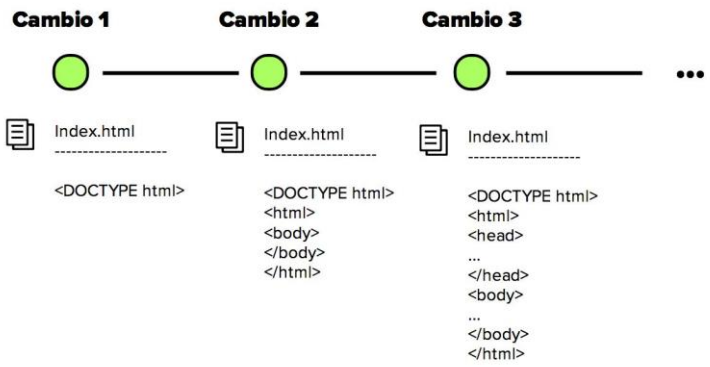
Tema 1

Qué es y que no es GIT?

- Git es un sistema de control de versiones distribuido creado por Linus Torvalds. Sirve para guardar el historial de cambios en archivos (normalmente código) y facilitar el trabajo en equipo.

- Git te permite:

  - Guardar versiones de tu proyecto (como checkpoints)
  - Volver atrás si algo se rompe
  - Colaborar con otras personas sin pisarse cambios
  - Trabajar en ramas (branches) para probar cosas sin afectar lo principal



Qué es y que no es GIT?

- Es muy común confundir Git con otras cosas.
  - **No es lo mismo que GitHub**
    GitHub es una plataforma online que *usa Git*, pero Git funciona sin internet.
  - **No es almacenamiento en la nube como Google Drive o Dropbox**
    Git guarda versiones, no archivos “para subir y descargar” como nube tradicional.
  - **No es un editor de código**
    No sirve para programar directamente (eso lo haces con editores como VS Code, etc.)
  - **No es solo para programadores (aunque es su uso principal)**
    Puede usarse para textos, diseños, etc., pero está optimizado para código.

✓ Git ES	❌ Git NO es
Sistema de control de versiones	Plataforma web
Historial de cambios	Almacenamiento tipo nube
Herramienta para colaborar	Editor de código
Software local (puede ser offline)	Algo que funcione solo con internet

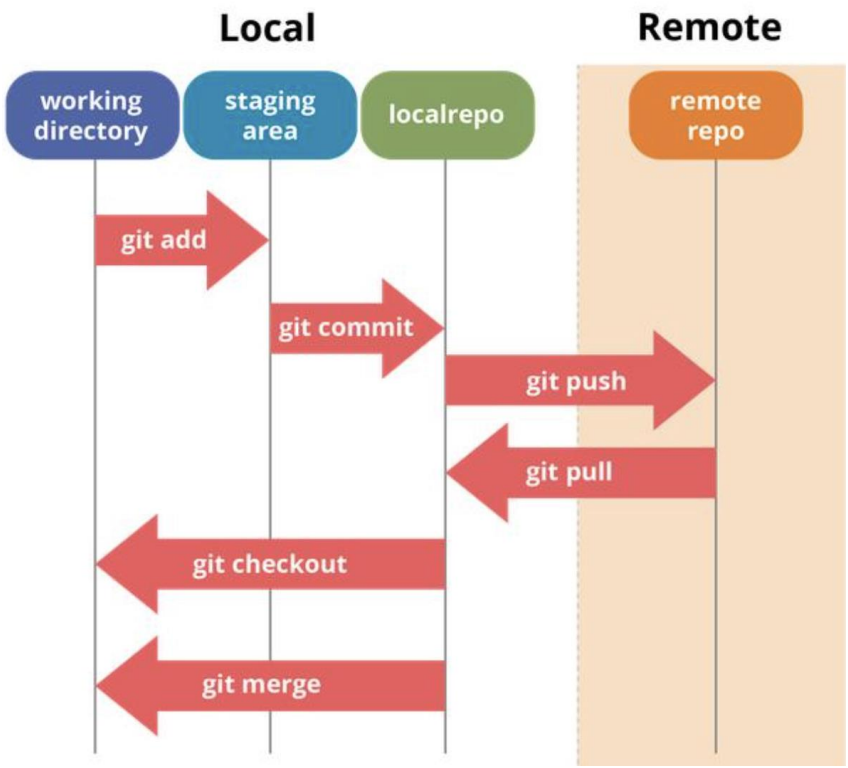
Cómo usar correctamente GIT

- Usar Git correctamente no es solo saber comandos, sino entender un flujo de trabajo limpio y seguro.

- El ciclo más importante es este:

  1. Modificas archivos
  2. Los preparas → git add
  3. Guardas versión → git commit
  4. Subes cambios → git push

  Y para traer cambios:
  - git pull



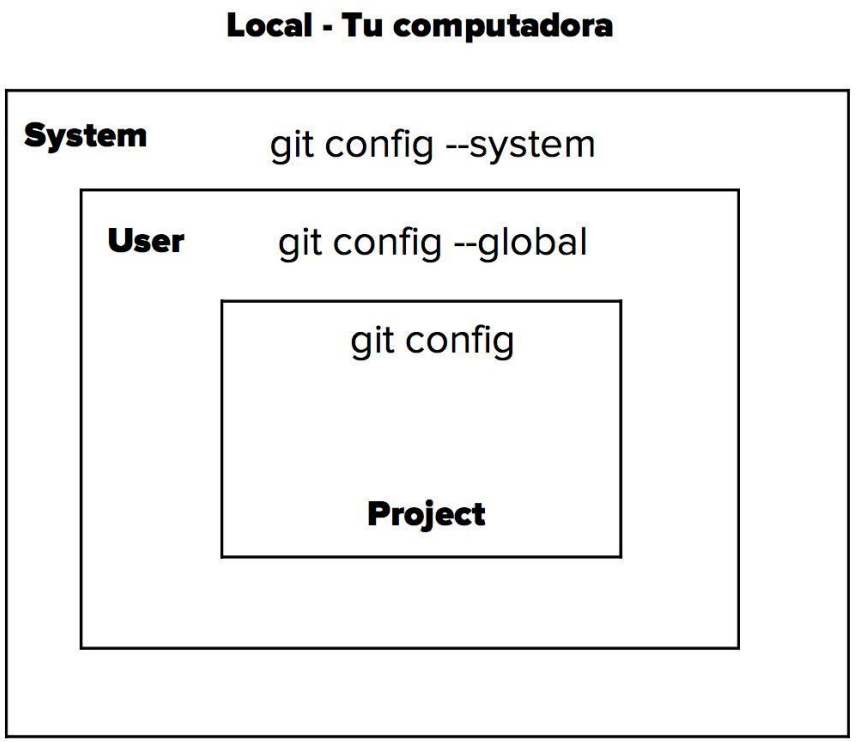
Cómo usar correctamente GIT

- **Buenas prácticas clave**:
  - Haz commits pequeños y frecuentes
  - Usa ramas para cada tarea
  - Escribe mensajes claros
  - Haz pull antes de trabajar
  - No subas archivos innecesarios (.gitignore)

- **Errores comunes:**
  - Hacer commits gigantes
  - Trabajar siempre en main
  - No hacer pull → conflictos
  - Subir archivos sensibles (contraseñas)
  - Usar git add . sin revisar

Configuración imprescindible

- Existen 3 niveles en tu equipo donde se desarrolla registros de configuración de GIT:



Configuración imprescindible

- **System**
  - Es una configuración para todos los usuarios de tu equipo. En la práctica, lo ideal es que no trabajes en este nivel realmente. Al crear preferencias de GIT en System, afectará todo el entorno de tu área local.
  - Regularmente es un archivo que se encuentra, dependiendo del sistema operativo, en: etc/gitconfig
  - Para hacer modificaciones al registro de GIT en System, debes utilizar en terminal:

```
git config --system [variable] “[descripción de variable]"
```

Configuración imprescindible

- **User**
  - Esta configuración es la más usada y recomendada. Aplica para el usuario de tu computadora, específicamente. Para encontrar este archivo, tienes que ir al “root” ó a la carpeta principal de usuario.
  - Para hacer modificaciones al registro de GIT en User, debes utilizar en terminal:

```
git config --global [variable] “[descripción de variable]"
```

Configuración imprescindible

- **Project**
  - Puedes generar una configuración específica para un proyecto puntual. Si por ejemplo, para ese proyecto necesitas utilizar otro correo ó un nombre diferente.
  - El archivo de gitconfig (configuraciones) para un proyecto específico lo puedes encontrar en:

    ~/[Ruta del proyecto]/.git/config

  - Para hacer modificaciones al registro de GIT en un proyecto específico, debes utilizar en terminal:

    git config [variable] “[descripción de variable]”

  - Como puedes observar, el git config se coloca directamente con la variable. Ya no utilizamos como en los anteriores algún --global ó –system

Configuración imprescindible

- Instalar GIT:

  https://git-scm.com/install/

  - Comprobar si se ha instalado correctamente: git --versión
  - git config --global user.name “TU NOMBRE"
  - git config --global user.email “TU CORREO DE GITHUB"
  - git config --global --list

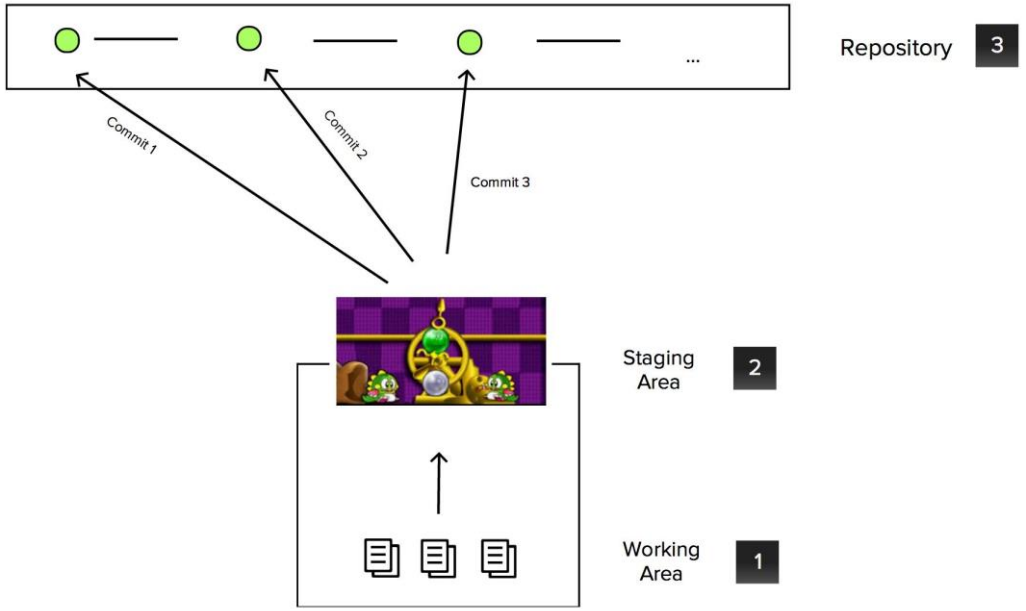
Fundamentos y arquitectura interna de GIT

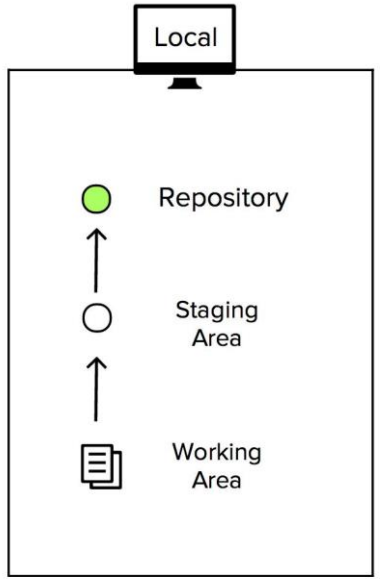
Tema 2

Diferencias entre el working directory, el stage y el repositorio

- Uno de los principales conceptos que debemos entender, antes de entrar en práctica, es la arquitectura de árbol de GIT.

- GIT va generando un historial de tu proyecto sobre un registro llamado repositorio. Sin embargo, para que podamos ir marcando cada “commit”, pasará por un proceso sencillo para organizarlo mejor.

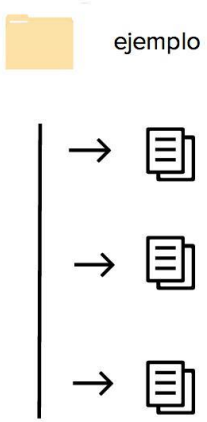




Diferencias entre el working directory, el stage y el repositorio

- **Escenario**

- Iniciamos la creación de un proyecto, nuestra carpeta base se llama “ejemplo”. Dentro de esta carpeta, crearemos 3 archivos.

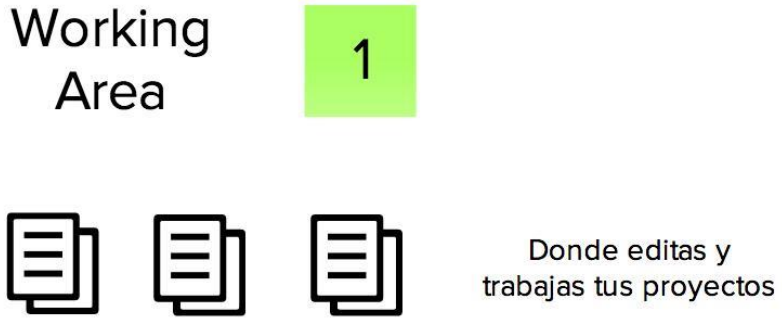


- Bien, una vez hecho esto, empezamos a explicar cómo se mueven estos archivos en cada ambiente.

Diferencias entre el working directory, el stage y el repositorio

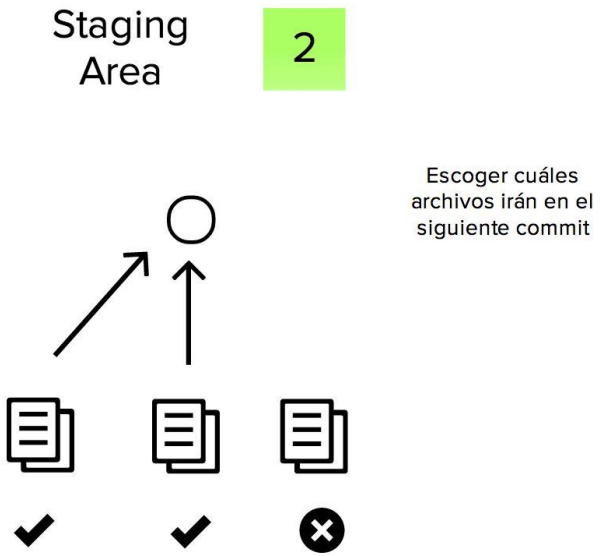
- **Área de trabajo (Working Area)**

- Es el área de GIT donde realizas modificaciones a tu proyecto. Git sabe que estás haciendo cambios y ediciones a esos archivos, pero él no los está registrando.



Diferencias entre el working directory, el stage y el repositorio

- **Área de preparación (Staging Area)**
- Ahora bien, imaginemos que estuvimos trabajando en los 3 archivos, pero uno de esos archivos sigue todavía muy mal y con errores.
- Los otros 2 está muy bien y queremos guardar específicamente los avances de esos archivos. Entonces, en esta área se selecciona qué archivos queremos que salgan en el siguiente “commit”.
- Podría decirse que en esta fase lo único que hacemos es elegir a los archivos que se encapsularán en el "commit".
- Si esta área no existiera, entonces tendrías que encapsular los 3 obligatoriamente. Y esto, a largo plazo, podría afectar la estabilidad, organización y agilidad del proyecto.

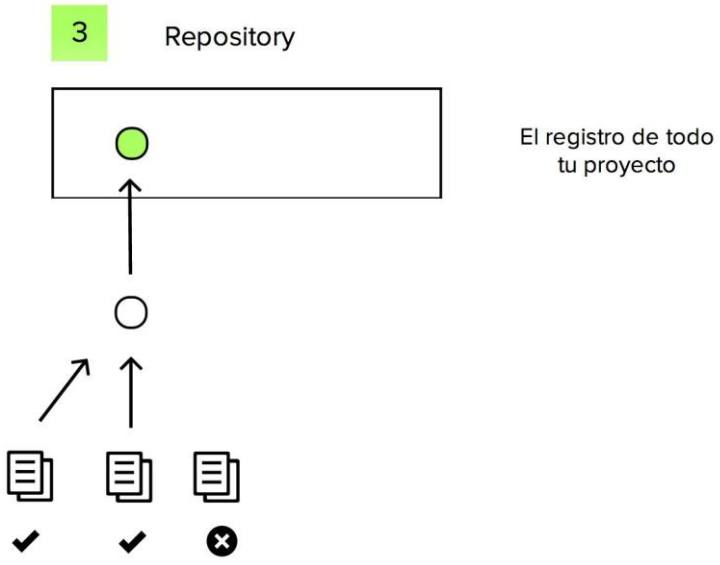


Diferencias entre el working directory, el stage y el repositorio

- **Repositorio (Repository)**

- El repositorio es el lugar donde se encuentra todo el historial del proyecto. Los diferentes cambios que han tenido los archivos, autorías, fechas, logs.

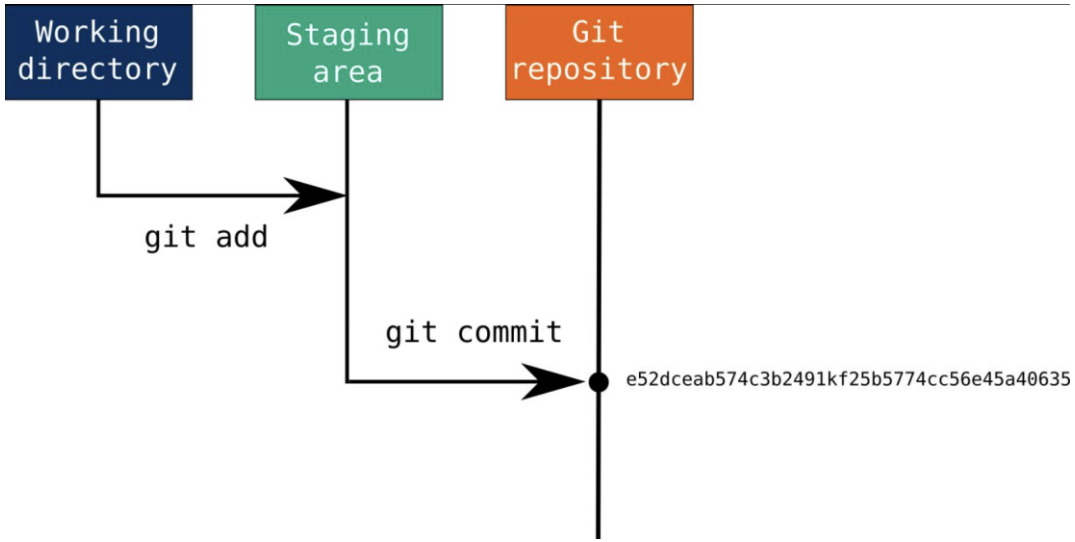
- En esta última área se imprime, registra y marca el nuevo cambio (commit).



Qué es exactamente un commit

- Un commit en Git es básicamente una foto (snapshot) de tu proyecto en un momento concreto. Es como decir: “Así está mi proyecto ahora, guarda este estado para siempre”.

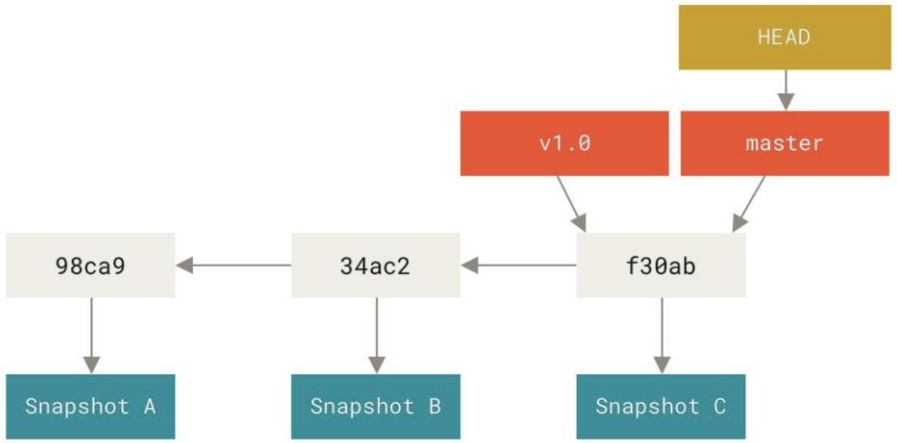
- Cada commit guarda:
  - Los cambios que hiciste en los archivos
  - Un mensaje descriptivo (lo escribes tú)
  - Autor (tú)
  - Fecha y hora
  - Un identificador único (hash)



Tipos de referencias (HEAD, ramas y tags)

- En Git, las referencias (refs) son nombres que apuntan a commits. Son la forma de moverte por el historial sin memorizar hashes largos.
- Los 3 tipos clave son: HEAD, ramas (branches) y tags.
  - **HEAD** es un puntero especial que indica: "En qué commit estás trabajando ahora mismo"
  - Normalmente apunta a una rama:

```
git checkout <hash>
```

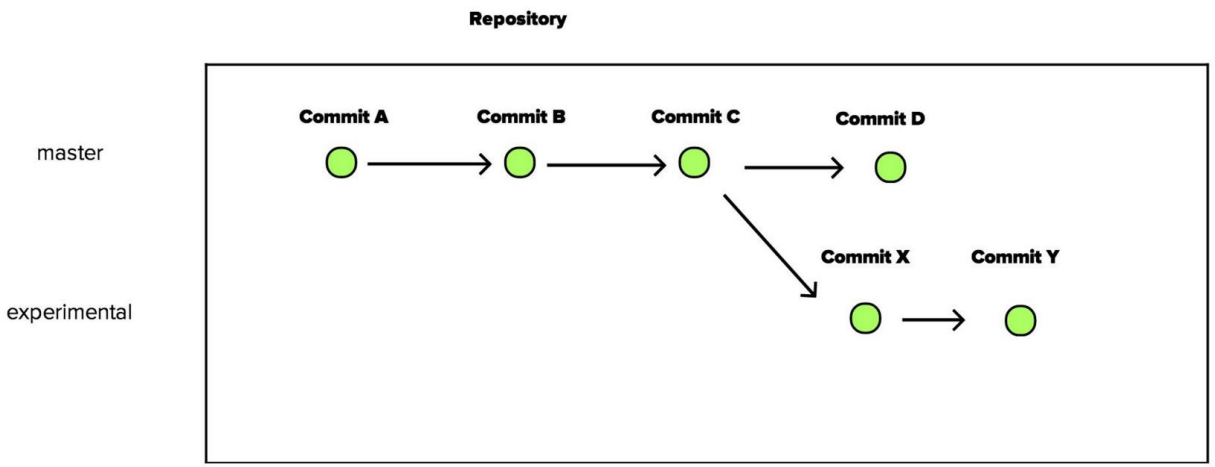


Tipos de referencias (HEAD, ramas y tags)

- Una **rama** es simplemente: Un puntero móvil a un commit
- Cuando haces un commit en una rama:
  - La rama se mueve hacia adelante
  - HEAD también

```
git branch nombre-rama
git checkout nombre-rama
git checkout -b nueva-rama
```

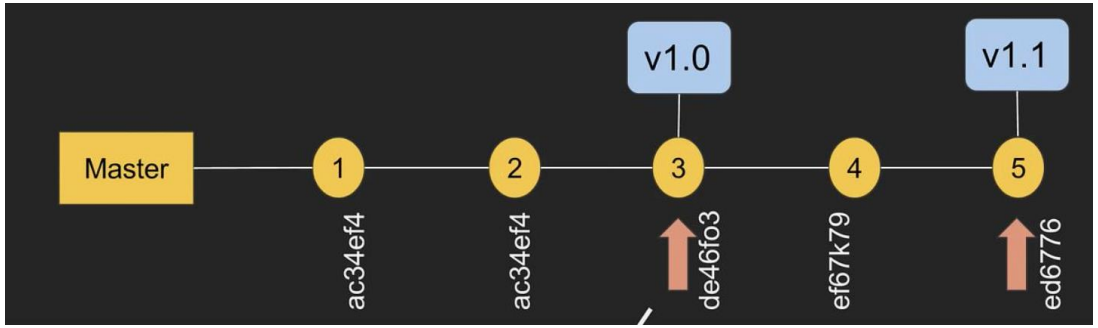
- Para qué sirven
  - Trabajar sin romper el código principal
  - Desarrollar features
  - Experimentar



Tipos de referencias (HEAD, ramas y tags)

- Un **tag** es: Un nombre fijo que apunta a un commit específico
- A diferencia de las ramas:
  - No se mueve
  - Es permanente
- Para qué sirven
  - Marcar versiones (v1.0, v2.0…)
  - Releases de software
  - Puntos importantes del proyecto
- Tipos de tags
  - Ligero (lightweight) → solo nombre
  - Anotado (annotated) → incluye autor, fecha, mensaje

```
git tag v1.0
git tag -a v1.0 -m "Versión 1.0"
```



Tipos de referencias (HEAD, ramas y tags)

- Cómo se relacionan todos:
  - **HEAD** → dónde estás
  - **Rama** → línea de trabajo que avanza
  - **Tag** → punto fijo en el tiempo

Referencia	Qué es	Se mueve
HEAD	Tu posición actual	✓
Rama	Línea de desarrollo	✓
Tag	Marca de versión ↓	✗

Commits

Tema 3

Cómo preparar selectivamente un commit

- Preparar un commit **selectivamente** en Git significa elegir **exactamente qué cambios entran en el commit**, en lugar de meter todo de golpe. Es una de las habilidades más importantes para trabajar de forma profesional.
- Añadir archivos específicos:

```
git add archivo1.js
git add archivo2.css
```

- Añadir partes de un archivo:

```
git add -p
```

- Esto abre modo interactivo:

```
Stage this hunk [y,n,q,a,d,s,e,?]?
```

  - y → añadir este cambio
  - n → saltarlo
  - s → dividir en partes más pequeñas
  - e → editar manualmente

Cómo preparar selectivamente un commit

- Ver qué estás preparando:

```
git status
git diff --staged
```

- Quitar cosas del staging:
  - **deshacer cambios en archivos**, sin tocar el historial. Quitar cambios no guardados

```
git restore archivo.txt
```

  - Quitar del staging (pero mantener cambios). Lo saca del commit, pero no borra el cambio

```
git restore --staged archivo.txt
```

Cuando hacer un commit

- Haz un commit cuando completes una unidad lógica de cambio Es decir:
  - una idea
  - una mejora
  - un arreglo
  - una tarea concreta

Cuando hacer un commit

- Casos claros donde **DEBES** hacer commit:

1. Cuando terminas algo pequeño pero completo
   - Ejemplos:
     - ✅ “Añade validación del formulario”
     - ✅ “Corrige bug en login”
     - ✅ “Cambia color del botón”

2. Antes de cambiar de tarea
   - Si estás haciendo login y luego pasas a otra cosa: Haz commit primero
     - ✅ Evita mezclar cambios
     - ✅ Mantiene el historial limpio

Cuando hacer un commit

3. Antes de hacer algo arriesgado. Haz commit como “punto de seguridad”
   - Ejemplo:
     - refactor grande
     - borrar código
     - probar algo incierto

4. Cuando tu código funciona
   - compila
   - no rompe nada importante
     - ✔ No tiene que ser perfecto, pero sí estable

5. Después de usar staging selectivo

Como deshacer commits

- Deshacer commits en Git se puede hacer de **varias maneras**, dependiendo de **qué quieres deshacer exactamente** y **si ya subiste el commit al repositorio remoto o no**.

- IMPORTANTE: Si le ponemos el hash del commit a borrar, ese no lo elimina, sino que borra desde ese hasta el final de la rama

1. Deshacer el último commit (sin perder cambios)

```
git reset --soft HEAD~1
```

- Resultado:
  - ❌ El commit desaparece
  - ✅ Los cambios siguen en staging

Como deshacer commits

2. Deshacer commit pero mantener cambios en archivos. Equivalente a --mixed

```
git reset HEAD~1
```

- Resultado:
  - ❌ El commit se elimina
  - ✅ Cambios siguen en el proyecto
  - ❌ Ya no están en staging

Como deshacer commits

3. Borrar commit y cambios completamente

```
git reset --hard HEAD~1
```

- Resultado:
  - ❌ El commit se elimina
  - ❌ Cambios en el proyecto

Como recuperar commits

- Los commits eliminados se pueden recuperar si tenemos el hash

```
git reset --soft HEAD~1
```

```
git reset --hard HEAD~1
```

```
git reset HEAD~1
```

Como deshacer commits

- Resumen:

Comando	Qué hace	Cuándo usar
`git reset --soft`	Quita commit pero deja staging	Rehacer commit
`git reset`	Quita commit y staging	Reorganizar cambios
`git reset --hard`	Borra todo	Error local

Buenas prácticas

- Un commit = una idea
- Mensajes claros y útiles
- Usa convenciones (muy importante en equipos). Una muy usada es **Conventional Commits**:
    - feat: → nueva funcionalidad
    - fix: → corrección de bug
    - refactor: → mejora sin cambiar comportamiento
    - docs: → documentación
    - style: → formato (espacios, etc.)
    - test: → tests

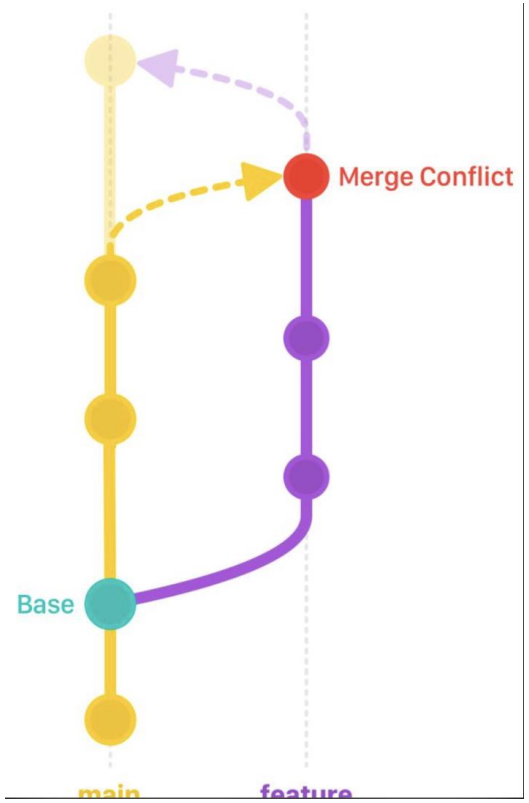
- Commits pequeños
- Código funcional
- Usa staging selectivo

Como resolver los conflictos

Tema 4

Qué es un conflicto

- Un conflicto ocurre cuando Git no puede decidir automáticamente cómo combinar cambios.
- Pasa cuando dos versiones modifican la misma parte del código.
- ¿Por qué ocurren?
    - Dos ramas modifican la misma línea
    - Se elimina/modifica el mismo archivo
    - Cambios incompatibles entre sí
    - Mucho tiempo sin sincronizar (git pull)



Como reducir su frecuencia

1. Haz pull frecuentemente. Mantienes tu rama actualizada
2. Haz commits pequeños. Cambios pequeños = conflictos pequeños
3. Usa ramas por tarea. Menos gente tocando lo mismo
4. Comunícate en equipo. Parece obvio, pero evita muchos conflictos
5. Evita archivos gigantes

Cómo resolverlos

- Cuando ocurre un conflicto, Git te avisa:

```
CONFLICT (content): Merge conflict in archivo.js
```

- Paso 1: Ver archivos en conflicto

```
git status
```

- Paso 2: Editar manualmente. Debes decidir quedarte con uno, combinar ambos o Reescribirlo

- Paso 3: Marcar como resuelto

```
git add archivo.js
```

- Paso 4: Finalizar

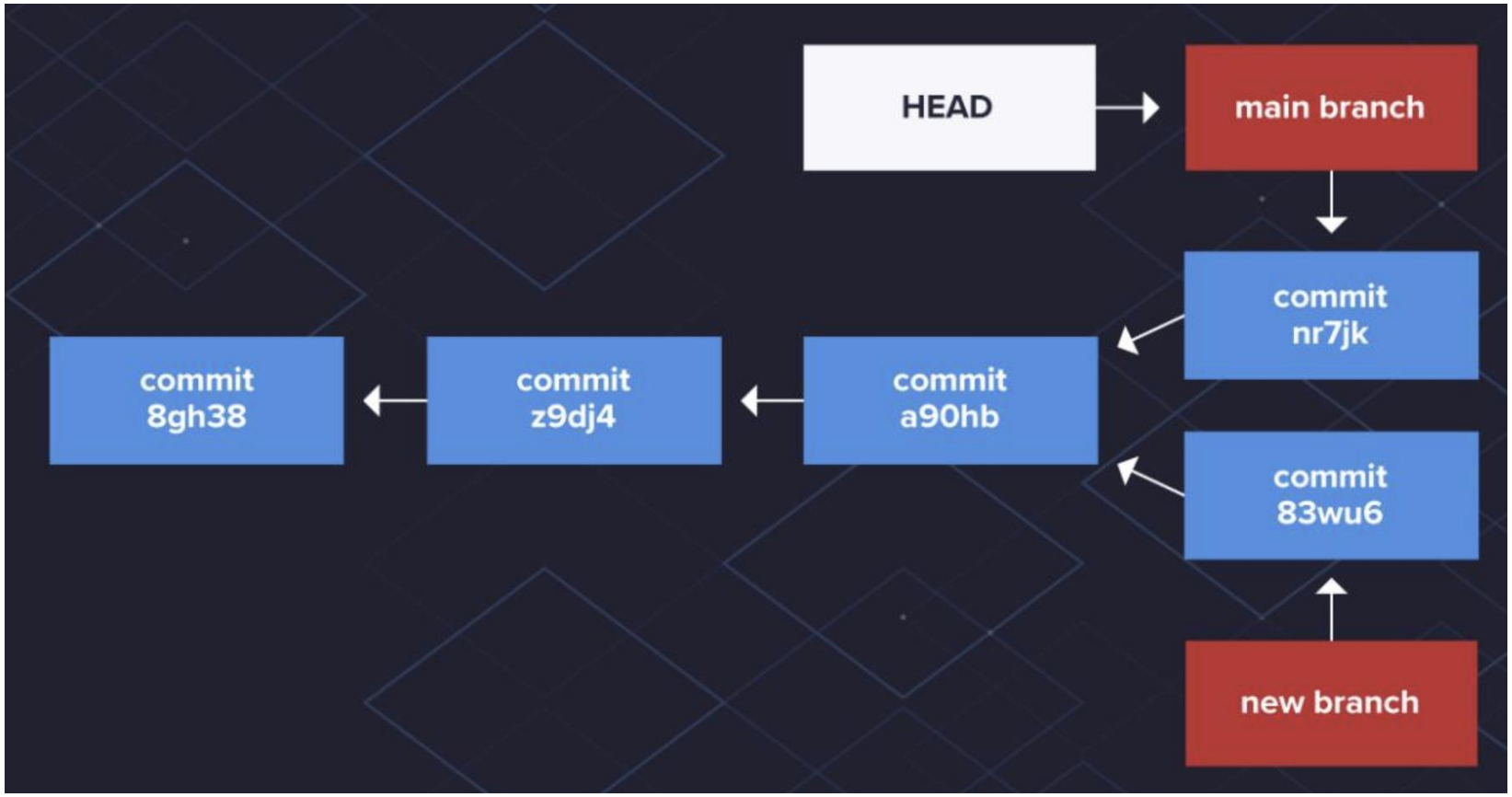
```
git commit
```

Branching

Tema 5

Qué es exactamente una rama en GIT

- Una rama (branch) en Git es un puntero (referencia) que apunta al último commit de una línea de trabajo

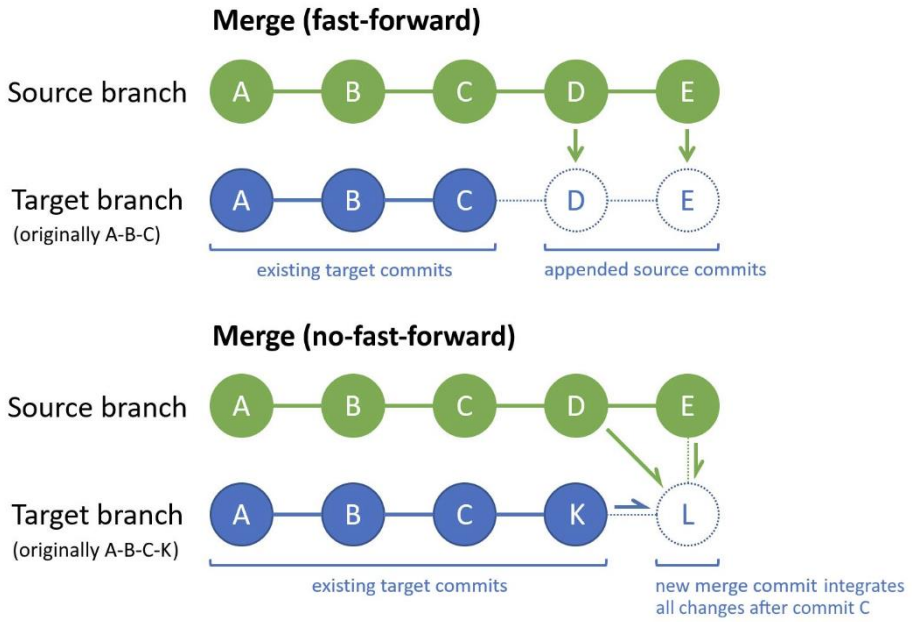


Cómo trabajar con ramas y diferentes maneras de fusionarlas

- Trabajar con ramas en Git es **la base del trabajo profesional en equipo**. La clave no es solo crearlas, sino saber **cómo integrarlas correctamente**.
- Para fusionar ramas podemos hacerlo de varias formas:
  - merge
  - rebase
  - cherry-pick
  - reverse

Cómo trabajar con ramas y diferentes maneras de fusionarlas

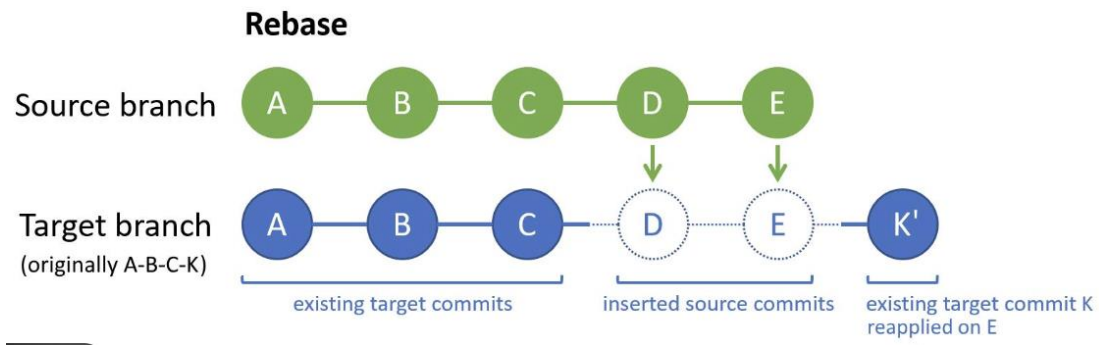
- **Merge (la forma clásica)**:
  - Une historiales
  - Puede crear un commit extra (merge commit)
    - Tipos de merge:
      - Fast-forward
      - Merge commit



Cómo trabajar con ramas y diferentes maneras de fusionarlas

- **Rebase (historial limpio)**
  - Reescribe tu rama como si hubiera nacido desde el último main

```
git rebase main
```

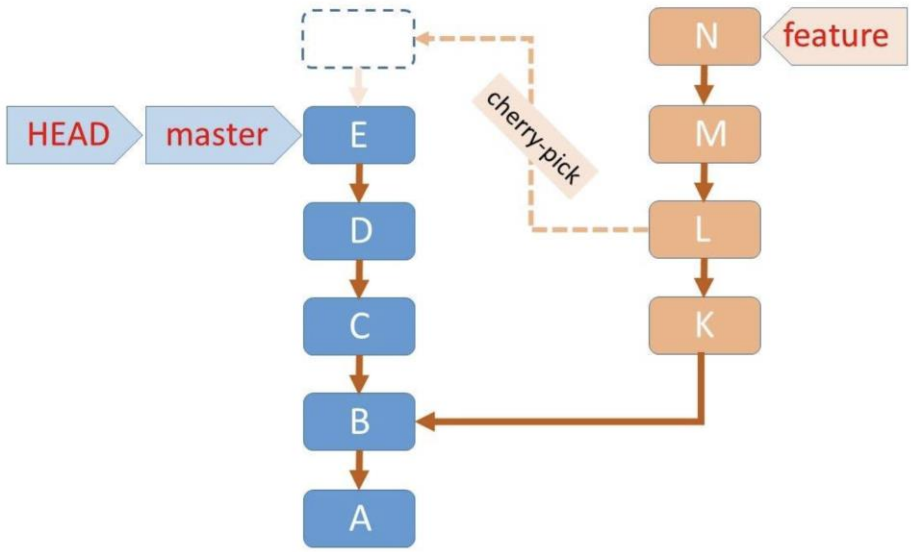


  - Cuándo NO usarlo
    - En ramas compartidas
    - Si ya hiciste push

Cómo trabajar con ramas y diferentes maneras de fusionarlas

- **cherry-pick (traer commits especificos):**
  - Trae un commit concreto de otra rama

```
git cherry-pick HASH
```

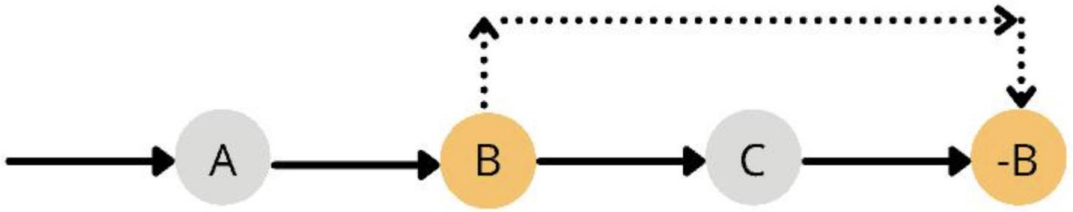


  - Cuándo usarlo:
    - Solo quieres un cambio específico
    - No toda la rama

Cómo trabajar con ramas y diferentes maneras de fusionarlas

- **Reverse (deshacer cambios):**
  - Crea un commit que deshace otro

```
git revert HASH
```

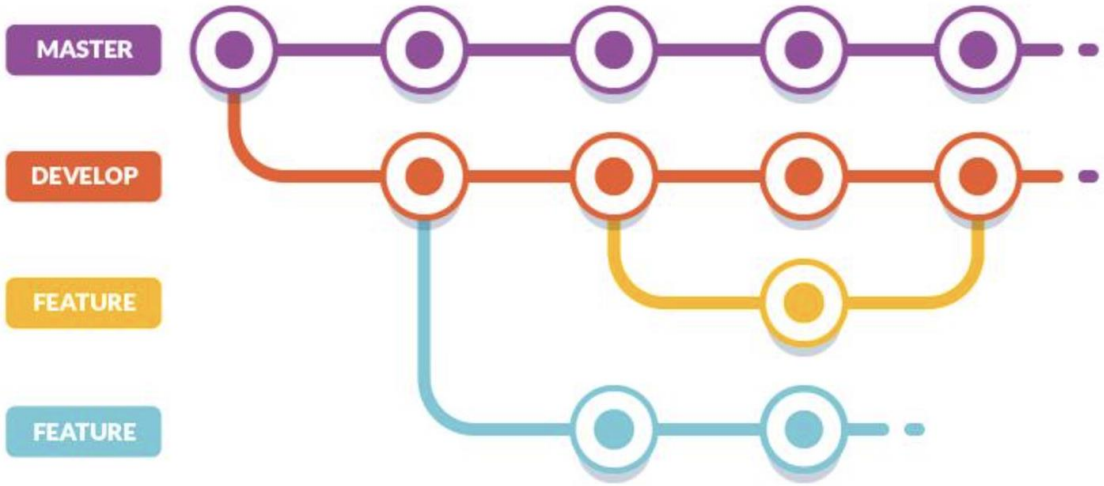


Modelos de branching

- Los modelos de branching son estrategias para organizar cómo usas ramas en Git en equipo. No hay uno único "correcto"; depende del tipo de proyecto y ritmo de desarrollo.
- Los más importantes son:
  - Git Flow
  - GitHub Flow
  - GitLab Flow
  - Trunk-Based Development

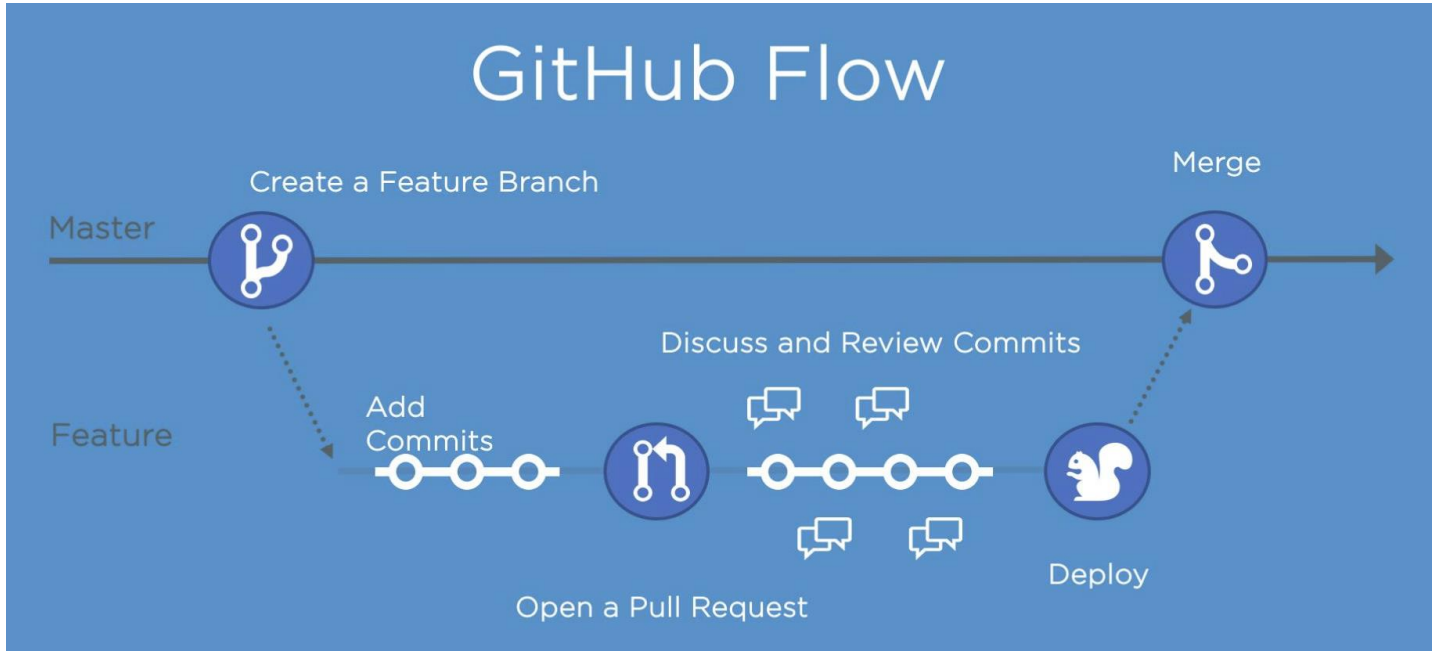
Git Flow (Modelo clásico)

- Es un modelo estructurado y completo
- Ramas principales:
  - main → producción
  - develop → desarrollo
  - feature/* → nuevas funcionalidades
  - release/* → preparación de versiones
  - hotfix/* → arreglos urgentes
- Cuándo usarlo:
  - Proyectos grandes
  - Equipos organizados
  - Versiones formales



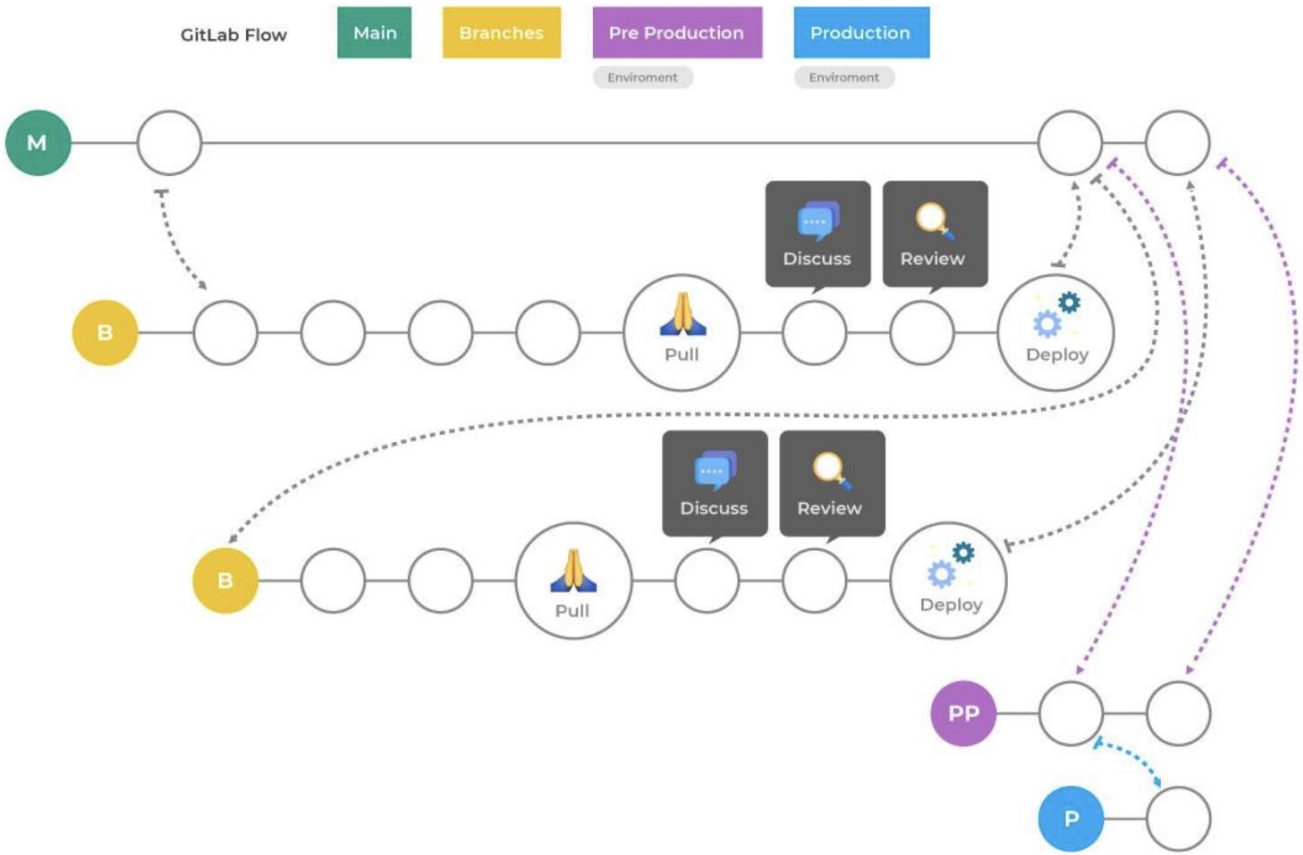
GitHub Flow (simple y moderno)

- Modelo simple y muy usado hoy en día
- Cómo funciona:
  - main siempre estable
  - Creas una rama (feature/...)
  - Trabajas ahí
  - Pull Request en GitHub
  - Merge a main
- Ventajas:
  - Simple
  - Rápido
  - Ideal para CI/CD



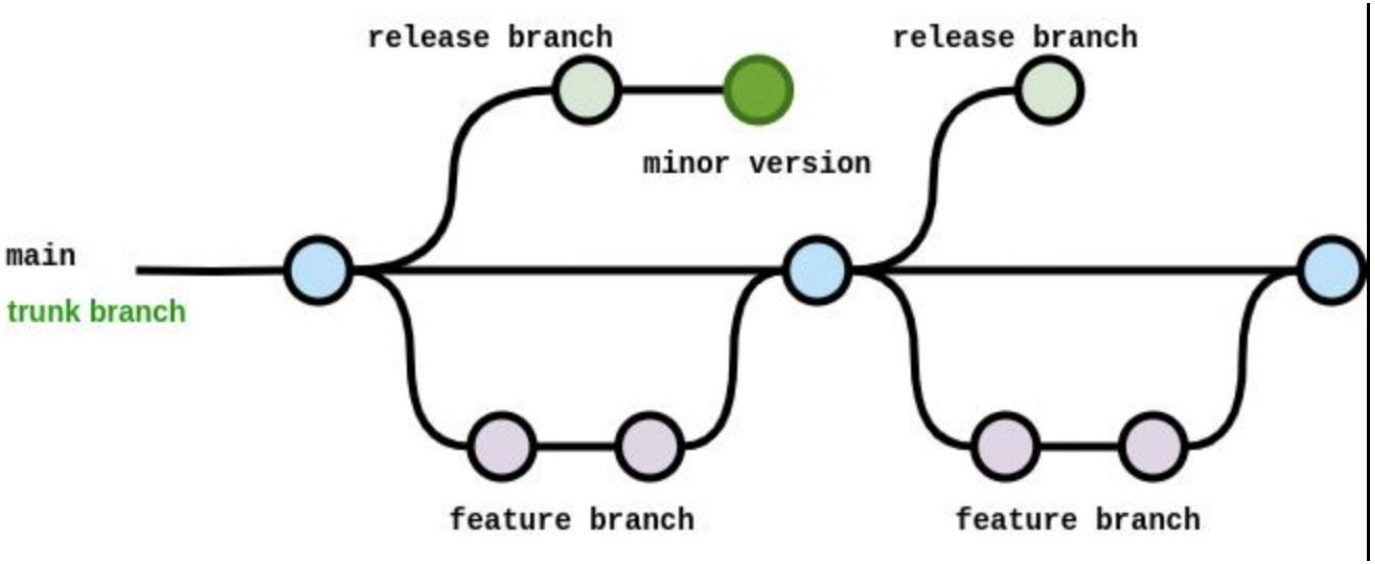
GitLab Flow

- Mezcla entre Git Flow y GitHub Flow
- Características:
  - Usa ramas por entorno:
    - main
    - staging
    - production
- Cuándo usarlo
  - Apps con varios entornos
  - DevOps / despliegues controlados



Trunk-Based Development

- Modelo moderno usado en empresas grandes
- Cómo funciona:
  - Todo gira en torno a main (trunk)
  - Ramas muy cortas (horas o días)
  - Integración constante
- Ventajas:
  - Menos conflictos
  - Integración rápida
  - Ideal para CI/CD



Trabajando con repositorios remotos

Tema 6

Como comunicar los repositorios locales con los remotos

- Tienes:
  - 💻 Repositorio local → tu máquina
  - ☁️ Repositorio remoto → servidor (GitHub, etc.)
- Y necesitas sincronizarlos.

```
git remote add origin URL
```

Diferentes escenarios cuando trabajamos en equipo contra un repositorio remoto

- Cuando trabajas en equipo contra un repositorio remoto (por ejemplo en GitHub), hay varios **escenarios típicos** que debes saber manejar para no romper nada y trabajar fluido.

1. Varios desarrolladores subiendo cambios:

   - Tu repo está desactualizado

```
! [rejected] (non-fast-forward)
```

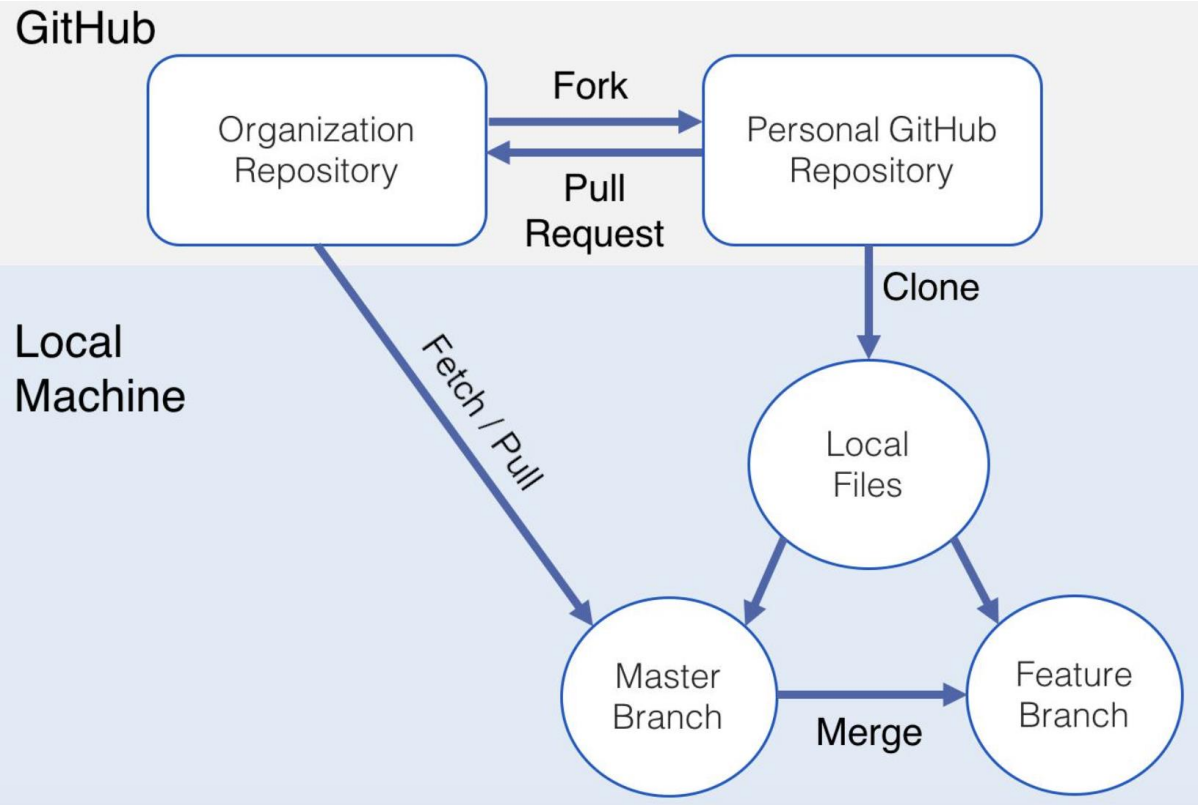
   - Primero sincronizas, luego subes

```
git pull --rebase
git push
```

Diferentes escenarios cuando trabajamos en equipo contra un repositorio remoto

2. Trabajo con ramas y Pull Requests

- Flujo típico:
  - Creas rama → feature/login
  - Trabajas ahí
  - git push
  - Abres PR en GitHub
  - Revisan
  - Merge a main
- Ventaja:
  - Código revisado
  - Menos errores

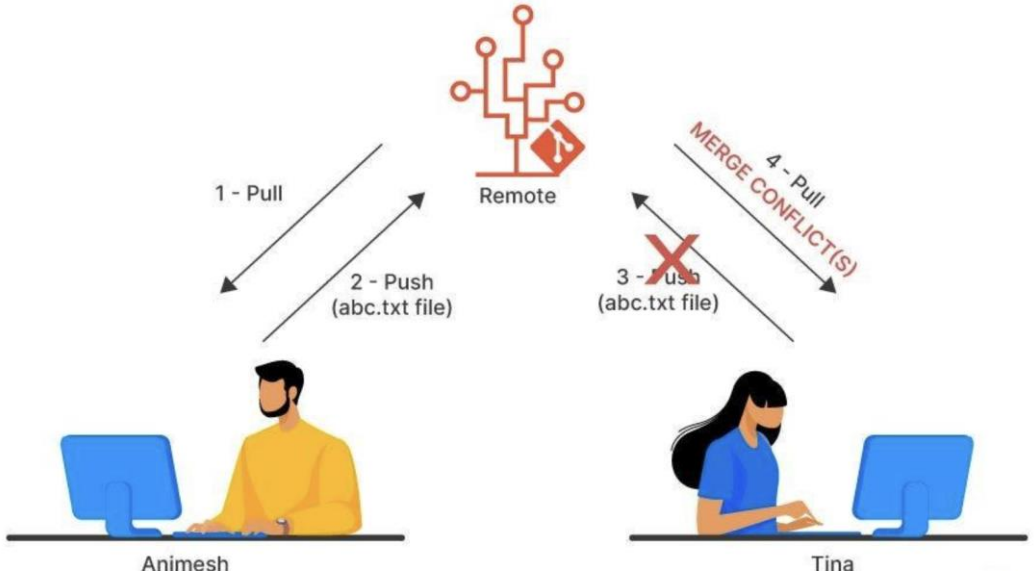


Diferentes escenarios cuando trabajamos en equipo contra un repositorio remoto

3. Conflictos al integrar cambios

- Qué pasa
  - Dos personas modifican lo mismo

- Solución
  - git pull
  - Resolver conflicto manualmente
  - git add
  - git commit

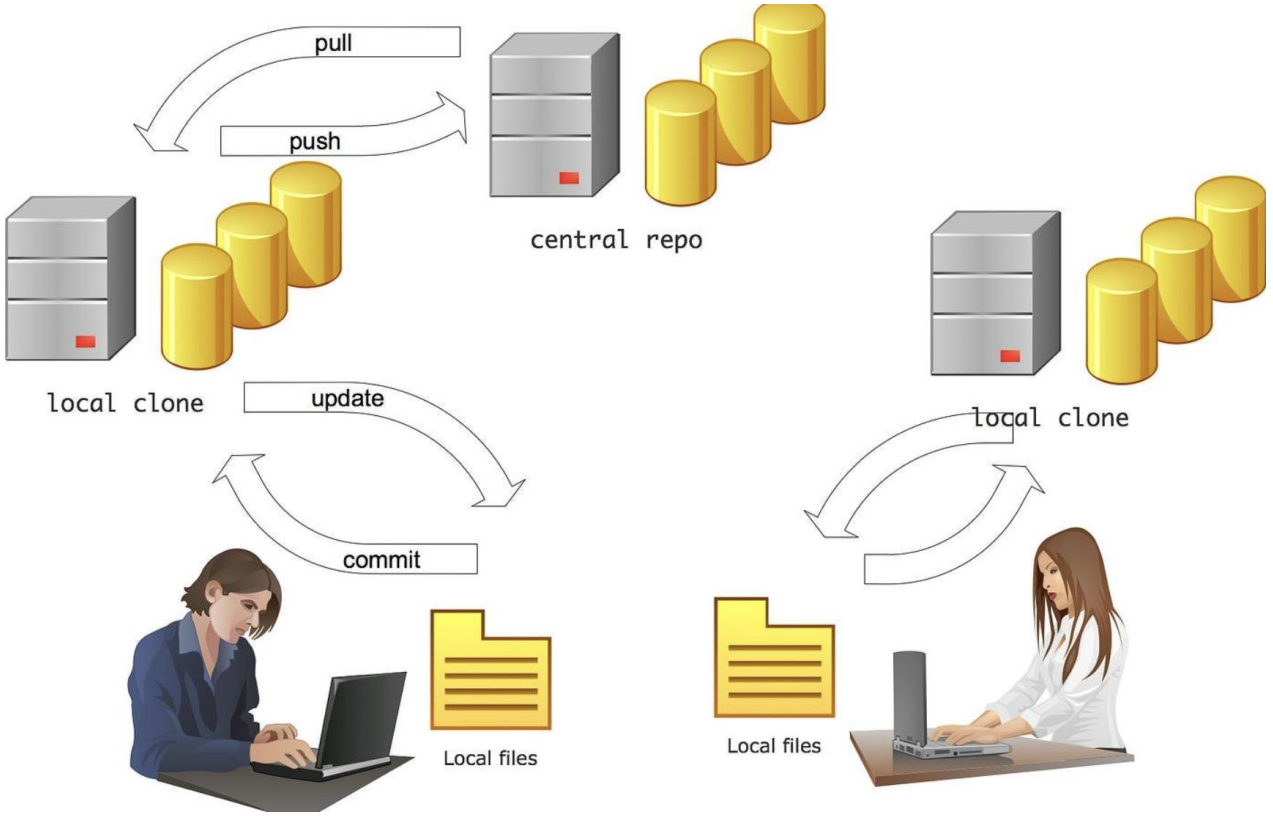


Diferentes escenarios cuando trabajamos en equipo contra un repositorio remoto

4. Sincronización constante (día a día)

- Flujo recomendado:

```
git pull
# trabajas
git add -p
git commit
git push
```



Configuración básica

- Crear el repositorio remoto:

```
https://github.com/usuario/proyecto.git
```

- Añadir remoto en tu proyecto local:

```
git remote add origin URL_DEL_REPO
```

- Ver remotos:

```
git remote -v
```

Pull Requests

Tema 7

Por qué usar PR

- Un **Pull Request (PR)** es un concepto central en el trabajo colaborativo con Git y plataformas como GitHub, GitLab o Bitbucket.
- Un **Pull Request** es una **solicitud para que tus cambios en una rama sean revisados y fusionados** en otra rama (normalmente main o develop) en un repositorio remoto.
- Ventajas de un Pull Request
  - Permite revisión de código antes de integrarlo
  - Facilita discusión sobre cambios
  - Reduce errores en main o develop
  - Permite CI/CD automático en la rama

Cómo hacer correctamente una PR

- Buenas prácticas
  - Haz PRs pequeños y enfocados (1 funcionalidad o bug fix)
  - Escribe descripción clara de qué cambia y por qué
  - Actualiza la rama antes de mergear (git pull --rebase)
  - Usa branch protection rules en main para obligar revisiones

Cómo configurar el repositorio remoto si usamos PR

1. Crear ramas para tus PR

   - Cada tarea o feature debe tener su propia rama:

```
git checkout -b feature/login
```

2. Preparar para PR

   - Mantén tu rama actualizada con main:

```
git fetch origin
git rebase origin/main
```

   - Sube los cambios si hay nuevos commits:

```
git push
```

   - Abre la PR desde la plataforma remota seleccionando:
     - Tu rama → feature/login
     - Destino → main o develop

Icono *Training Consulting*

Introducción a Git

Completa nuestra encuesta de satisfacción a través del QR

Curso Introducción a GIT (7- 10 abril)

GRACIAS

2026